

HEAP (Max-Heap)

Definizione

Albero binario **quasi completo** che soddisfa la **proprietà di heap**:

$$A[\text{Parent}(i)] \geq A[i] \quad \text{per ogni nodo } i$$

Rappresentazione Array

```
Parent(i) =  $\lfloor i/2 \rfloor$   
Left(i) =  $2i$   
Right(i) =  $2i + 1$ 
```

Altezza heap con n elementi: $h = \lfloor \log n \rfloor$

Operazioni Fondamentali

MaxHeapify - Mantenimento Proprietà

Precondizione: Sottoalberi sinistro e destro sono max-heap

Postcondizione: Sottoalbero radicato in i è max-heap

```
MaxHeapify(A, i):  
    l = Left(i)  
    r = Right(i)  
    max = i  
  
    if l ≤ heapsize AND A[l] > A[max]:  
        max = l  
    if r ≤ heapsize AND A[r] > A[max]:  
        max = r  
  
    if max ≠ i:  
        swap(A[i], A[max])  
        MaxHeapify(A, max)
```

Complessità: $O(\log n) = O(h)$

BuildMaxHeap - Costruzione Heap

```
BuildMaxHeap(A, n):  
    heapsize = n
```

```

for i = ⌊n/2⌋ downto 1:
    MaxHeapify(A, i)

```

Complessità: $O(n)$ - NON $O(n \log n)$!

Dimostrazione $O(n)$:

```

Nodi ad altezza h:  $\leq \lceil n/2^{(h+1)} \rceil$ 
Costo MaxHeapify ad altezza h:  $O(h)$ 

 $\sum_{h=0 \text{ to } \log n} \lceil n/2^{(h+1)} \rceil \cdot O(h) = O(n \cdot \sum_{h=0 \text{ to } \infty} h/2^h)$ 
 $= O(n \cdot 2) = O(n)$ 

```

HeapSort

```

HeapSort(A, n):
    BuildMaxHeap(A, n)
    for i = n downto 2:
        swap(A[1], A[i])
        heapsize = heapsize - 1
        MaxHeapify(A, 1)

```

Complessità: $\Theta(n \log n)$

Spazio: $O(1)$ - in-place

Code di Priorità

HeapMaximum

```

HeapMaximum(A):
    return A[1]

```

Complessità: $\Theta(1)$

HeapExtractMax

```

HeapExtractMax(A):
    if heapsize < 1:
        error "heap underflow"
    max = A[1]
    A[1] = A[heapsize]
    heapsize = heapsize - 1
    MaxHeapify(A, 1)
    return max

```

Complessità: $O(\log n)$

HeapIncreaseKey

```
HeapIncreaseKey(A, i, key):  
    if key < A[i]:  
        error "new key is smaller"  
    A[i] = key  
    while i > 1 AND A[Parent(i)] < A[i]:  
        swap(A[i], A[Parent(i)])  
        i = Parent(i)
```

Complessità: $O(\log n)$

MaxHeapInsert

```
MaxHeapInsert(A, key):  
    heapsize = heapsize + 1  
    A[heapsize] =  $-\infty$   
    HeapIncreaseKey(A, heapsize, key)
```

Complessità: $O(\log n)$



ALBERI BINARI DI RICERCA (ABR)

Definizione

Albero binario con **proprietà ABR**:

```
Per ogni nodo x:  
- y nel sottoalbero sinistro →  $y.key \leq x.key$   
- y nel sottoalbero destro →  $y.key \geq x.key$ 
```

Proprietà Fondamentale (InOrder)

Teorema: Un albero binario è ABR $\Leftrightarrow$ la visita simmetrica elenca i nodi in ordine crescente.

Dimostrazione ($\Leftarrow$): Per induzione sulla struttura dell'albero.

Operazioni - Complessità $O(h)$

TreeSearch - Ricerca

```
TreeSearch(x, k):  
    if x = nil OR k = x.key:  
        return x
```

```
if k < x.key:
    return TreeSearch(x.left, k)
else:
    return TreeSearch(x.right, k)
```

Versione iterativa (preferibile):

```
IterativeTreeSearch(x, k):
    while x ≠ nil AND k ≠ x.key:
        if k < x.key:
            x = x.left
        else:
            x = x.right
    return x
```

Complessità: $O(h)$

TreeMinimum / TreeMaximum

```
TreeMinimum(x):
    while x.left ≠ nil:
        x = x.left
    return x

TreeMaximum(x):
    while x.right ≠ nil:
        x = x.right
    return x
```

Complessità: $O(h)$

TreeSuccessor - Successore

```
TreeSuccessor(x):
    if x.right ≠ nil:
        return TreeMinimum(x.right)
    y = x.p
    while y ≠ nil AND x = y.right:
        x = y
        y = y.p
    return y
```

Complessità: $O(h)$

TreeInsert - Inserimento

```

TreeInsert(T, z):
    y = nil
    x = T.root
    while x ≠ nil:
        y = x
        if z.key < x.key:
            x = x.left
        else:
            x = x.right
    z.p = y
    if y = nil:
        T.root = z
    else if z.key < y.key:
        y.left = z
    else:
        y.right = z

```

Complessità: $O(h)$

TreeDelete - Cancellazione

```

TreeDelete(T, z):
    if z.left = nil:
        Transplant(T, z, z.right)
    else if z.right = nil:
        Transplant(T, z, z.left)
    else:
        y = TreeMinimum(z.right) // successore
        if y.p ≠ z:
            Transplant(T, y, y.right)
            y.right = z.right
            y.right.p = y
        Transplant(T, z, y)
        y.left = z.left
        y.left.p = y

Transplant(T, u, v):
    if u.p = nil:
        T.root = v
    else if u = u.p.left:
        u.p.left = v
    else:
        u.p.right = v
    if v ≠ nil:
        v.p = u.p

```

Complessità: $O(h)$

Prestazioni

- **Caso migliore:** $h = \Theta(\log n)$ - albero bilanciato
- **Caso peggiore:** $h = \Theta(n)$ - albero degenerato (lista)

RED-BLACK TREES

Definizione

ABR con **5 proprietà**:

1. Ogni nodo è **rosso** o **nero**
2. La radice è **nera**
3. Ogni foglia (nil) è **nera**
4. Se un nodo è rosso, entrambi i figli sono **neri**
5. Per ogni nodo, tutti i cammini verso le foglie contengono lo **stesso numero di nodi neri**

Black-Height

$bh(x)$ = numero di nodi neri su cammino da x (escluso) a foglia (inclusa)

Teorema Fondamentale

Un RB-Tree con n nodi interni ha altezza $h \leq 2 \log_2(n+1)$.

Dimostrazione:

Lemma: Sottualbero radicato in x ha $\geq 2^{bh(x)} - 1$ nodi interni
Prova per induzione su altezza di x .

Caso base: $h(x) = 0 \rightarrow x$ è foglia $\rightarrow bh(x) = 0 \rightarrow 2^0 - 1 = 0 \checkmark$

Passo induttivo:

- Figli di x hanno $bh \geq bh(x) - 1$
- Per ipotesi induttiva: $\geq 2^{bh(x)-1} - 1$ nodi ciascuno
- Totale $\geq 2 \cdot (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1 \checkmark$

Dalla proprietà 4: $bh(\text{radice}) \geq h/2$

Quindi: $n \geq 2^{h/2} - 1$

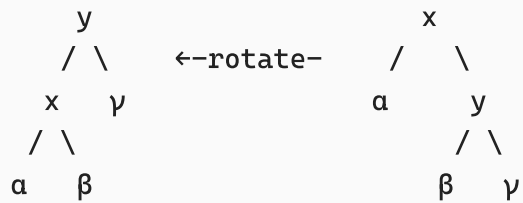
$$n + 1 \geq 2^{h/2}$$

$$\log_2(n+1) \geq h/2$$

$$h \leq 2 \log_2(n+1)$$

Rotazioni

Left-Rotate



```
LeftRotate(T, x):
    y = x.right
    x.right = y.left
    if y.left ≠ nil:
        y.left.p = x
    y.p = x.p
    if x.p = nil:
        T.root = y
    else if x = x.p.left:
        x.p.left = y
    else:
        x.p.right = y
    y.left = x
    x.p = y
```

Complessità: $O(1)$

Right-Rotate

Simmetrico a Left-Rotate.

RB-Insert

```
RBInsert(T, z):
    // Inserimento normale ABR
    y = nil
    x = T.root
    while x ≠ nil:
        y = x
        if z.key < x.key:
            x = x.left
        else:
            x = x.right
    z.p = y
    if y = nil:
        T.root = z
    else if z.key < y.key:
        y.left = z
    else:
```

```
y.right = z

z.left = nil
z.right = nil
z.color = RED
RBInsertFixup(T, z)
```

RB-Insert-Fixup

Violazioni possibili dopo inserimento:

- Proprietà 2 (radice nera)
- Proprietà 4 (padre rosso → figlio nero)

```
RBInsertFixup(T, z):
    while z.p.color = RED:
        if z.p = z.p.p.left:
            y = z.p.p.right // zio
            if y.color = RED:
                // Caso 1: zio rosso
                z.p.color = BLACK
                y.color = BLACK
                z.p.p.color = RED
                z = z.p.p
            else:
                if z = z.p.right:
                    // Caso 2: z figlio destro
                    z = z.p
                    LeftRotate(T, z)
                // Caso 3: z figlio sinistro
                z.p.color = BLACK
                z.p.p.color = RED
                RightRotate(T, z.p.p)
        else:
            // Simmetrico (scambia left e right)
    T.root.color = BLACK
```

Complessità: $O(\log n)$ - al massimo 2 rotazioni

RB-Delete

Simile a TreeDelete ma più complesso. **Complessità:** $O(\log n)$ - al massimo 3 rotazioni



Operazione	Heap	ABR	RB-Tree
Search	$O(n)$	$O(h)$	$O(\log n)$
Min/Max	$O(1)$	$O(h)$	$O(\log n)$
Insert	$O(\log n)$	$O(h)$	$O(\log n)$
Delete	$O(\log n)$	$O(h)$	$O(\log n)$
ExtractMin	$O(\log n)$	$O(h)$	$O(\log n)$
BuildHeap	$O(n)$	-	-
h in caso peggiore	$\log n$	n	$\log n$

CONSIGLI PER L'ESAME

Heap

- Ricorda: BuildMaxHeap è $O(n)$, NON $O(n \log n)$
- MaxHeapify richiede che i sottoalberi siano già heap
- Indice array: parte da 1, non da 0

ABR

- Sempre verificare proprietà ABR in esame
- Cancellazione: 3 casi (0, 1, 2 figli)
- Successore: se ha figlio destro $\rightarrow$ minimo destro, altrimenti risali

RB-Trees

- 5 proprietà: impararle a memoria
- Black-height: conta solo nodi neri, escluso nodo stesso
- Insert-Fixup: 3 casi, max 2 rotazioni
- Altezza garantita: $h \leq 2 \log_2(n+1)$